
Neural Networks by Hand

From Neurons to Backpropagation and Matrix Calculus

All mathematical equations are typeset as native LaTeX in the source and PDF.

This document provides a rigorous, step-by-step mathematical treatment of feed-forward neural networks, forward propagation, loss functions, computational graphs, backpropagation, gradient descent, and the matrix calculus behind modern deep learning.

Core thesis: A neural network is a parameterized function composition; the loss function maps predictions to a scalar objective; backpropagation applies reverse-mode differentiation to obtain exact parameter gradients; gradient descent uses those gradients to update parameters.

Learning goals By the end of this document, you should be able to compute forward passes and backward passes by hand, derive the vectorized recurrence

$$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot f' \left(z^{(l)} \right),$$

verify matrix shapes at every layer, and connect scalar chain-rule derivatives to full matrix outer products.

Contents

1	The Objective	4
2	The Artificial Neuron	4
2.1	Weights	4
2.2	Bias	5
3	Activation Functions	5
3.1	ReLU	5
3.2	Sigmoid	5
3.3	Hyperbolic Tangent (Tanh)	5
4	Necessity of Nonlinearity	5
5	Layer Formulation and Matrix Shapes	6
5.1	Matrix Dimensions	6
6	Forward Propagation	6
7	Loss Functions	6
8	Computational Graph and Chain Rule	7
9	Worked Single-Neuron Example	7
10	Backpropagation versus Gradient Descent	8
11	First-Order Taylor Justification	8
12	Vectorized Backpropagation Equations	8
13	Deriving the Weight Matrix Outer Product	9
14	Deriving the Hidden-Layer Recurrence	9
15	Worked Two-Layer Network by Hand	9
15.1	Forward Pass	10
15.2	Backward Pass	10
16	Jacobian and Vector-Jacobian Products	10
17	Softmax and Cross-Entropy Derivative	10

18 Mini-Batch Gradients and Optimizers	11
18.1 Standard Parameter Update Rules	11
19 Numerical Gradient Checking and Pathologies	11
19.1 Centered Finite Differences	11
19.2 Vanishing and Exploding Gradients	11
20 Procedure and Practice Exercise	12
20.1 7-Step Hand-Solving Sequence	12
20.2 Practice Problem	12
21 Core Equation Reference	12

1 The Objective

A neural network is a parameterized function. Given an input vector $\mathbf{x}$, parameters θ , and target $\mathbf{y}$, the network produces a prediction:

$$\hat{\mathbf{y}} = f(\mathbf{x}; \theta).$$

A loss function measures the discrepancy between the prediction and the target:

$$\mathcal{L} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}).$$

Training means finding parameters that minimize this scalar loss:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(f(\mathbf{x}; \theta), \mathbf{y}).$$

Everything else in this document answers one central question: **how do we calculate the gradient $\nabla_{\theta} \mathcal{L}$ required to improve θ ?**

Central distinction:

- **Backpropagation** is the algorithm that computes the gradient $\nabla_{\theta} \mathcal{L}$ efficiently.
- **Gradient-based optimization** is the procedure that updates θ using $\nabla_{\theta} \mathcal{L}$.

2 The Artificial Neuron

The fundamental building block of a feed-forward network is an affine transformation followed by a nonlinear activation. Given an input vector $\mathbf{x} \in \mathbb{R}^n$ and weight vector $\mathbf{w} \in \mathbb{R}^n$,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix},$$

the pre-activation scalar z is

$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b.$$

The neuron then applies a scalar activation function f :

$$a = f(z).$$

Single-neuron equation:

$$a = f(\mathbf{w}^T \mathbf{x} + b).$$

2.1 Weights

Each weight controls the sensitivity of the pre-activation to its corresponding input feature. For two inputs,

$$z = w_1 x_1 + w_2 x_2 + b.$$

If $w_1 > 0$ is large, increasing x_1 strongly increases z . If $w_1 < 0$, increasing x_1 decreases z .

2.2 Bias

The bias b shifts the affine hyperplane. Without the bias ($z = \mathbf{w}^T \mathbf{x}$), the decision boundary $\mathbf{w}^T \mathbf{x} = 0$ is forced to pass through the origin $\mathbf{x} = \mathbf{0}$. Including b allows the neuron to translate the activation threshold anywhere in input space.

3 Activation Functions

Activation functions introduce nonlinearity into the network.

3.1 ReLU

The Rectified Linear Unit (ReLU) is defined as

$$f(z) = \max(0, z) = \begin{cases} 0, & z < 0, \\ z, & z \geq 0. \end{cases}$$

Its derivative is

$$f'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

At $z = 0$, the subgradient is typically set to 0 in software implementations.

3.2 Sigmoid

The logistic sigmoid maps any real number into $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Its derivative satisfies the self-referential identity

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

If $a = \sigma(z)$, then $\frac{da}{dz} = a(1 - a)$.

3.3 Hyperbolic Tangent (Tanh)

The hyperbolic tangent maps real numbers into $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

Its derivative is

$$\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z) = 1 - a^2.$$

4 Necessity of Nonlinearity

Suppose we stack two purely linear layers without an activation function:

$$\mathbf{h} = W^{(1)} \mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{y} = W^{(2)} \mathbf{h} + \mathbf{b}^{(2)}.$$

Substituting $\mathbf{h}$ into the second equation yields

$$\begin{aligned}\mathbf{y} &= W^{(2)} \left(W^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} \\ &= \underbrace{\left(W^{(2)} W^{(1)} \right)}_{W'} \mathbf{x} + \underbrace{\left(W^{(2)} \mathbf{b}^{(1)} + \mathbf{b}^{(2)} \right)}_{\mathbf{b}'} \\ &= W' \mathbf{x} + \mathbf{b}'.\end{aligned}$$

Any composition of affine transformations is still a single affine transformation. Without nonlinear activations between layers, a deep network collapses algebraically into a single linear layer.

5 Layer Formulation and Matrix Shapes

For layer $l \in \{1, \dots, L\}$, let $\mathbf{a}^{(0)} = \mathbf{x}$. The layer recurrence is:

Forward layer equations:

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}).$$

5.1 Matrix Dimensions

If layer $l-1$ has n_{l-1} units and layer l has n_l units, the dimensions of every quantity are fixed:

$$W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}, \quad \mathbf{a}^{(l-1)} \in \mathbb{R}^{n_{l-1} \times 1}, \quad \mathbf{b}^{(l)} \in \mathbb{R}^{n_l \times 1}, \quad \mathbf{z}^{(l)}, \mathbf{a}^{(l)} \in \mathbb{R}^{n_l \times 1}.$$

Checking matrix dimensions at every step is the fastest way to verify a gradient derivation.

6 Forward Propagation

Forward propagation evaluates the network from input $\mathbf{x}$ to prediction $\hat{\mathbf{y}}$. For a two-layer network:

$$\begin{aligned}\mathbf{z}^{(1)} &= W^{(1)} \mathbf{x} + \mathbf{b}^{(1)}, \\ \mathbf{a}^{(1)} &= f^{(1)}(\mathbf{z}^{(1)}), \\ \mathbf{z}^{(2)} &= W^{(2)} \mathbf{a}^{(1)} + \mathbf{b}^{(2)}, \\ \hat{\mathbf{y}} &= f^{(2)}(\mathbf{z}^{(2)}).\end{aligned}$$

Written as a single nested composition:

$$\hat{\mathbf{y}} = f^{(2)} \left(W^{(2)} f^{(1)} \left(W^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} \right).$$

Forward propagation evaluates this nested function; backpropagation differentiates it.

7 Loss Functions

To train the parameters, prediction quality is summarized into a scalar loss $\mathcal{L}$. For scalar regression with prediction a and target y , squared error is

$$\mathcal{L} = \frac{1}{2}(a - y)^2, \quad \frac{\partial \mathcal{L}}{\partial a} = a - y.$$

For a vector output $\mathbf{a} \in \mathbb{R}^K$ and target $\mathbf{y} \in \mathbb{R}^K$,

$$\mathcal{L} = \frac{1}{2} \sum_{i=1}^K (a_i - y_i)^2 = \frac{1}{2} \|\mathbf{a} - \mathbf{y}\|_2^2, \quad \nabla_{\mathbf{a}} \mathcal{L} = \mathbf{a} - \mathbf{y}.$$

8 Computational Graph and Chain Rule

Consider a single neuron with squared error loss:

$$z = wx + b, \quad a = f(z), \quad \mathcal{L} = \frac{1}{2}(a - y)^2.$$

The dependency chain from parameters to loss is

$$(w, x, b) \longrightarrow z \longrightarrow a \longrightarrow \mathcal{L}.$$

By the single-variable chain rule along the path $w \rightarrow z \rightarrow a \rightarrow \mathcal{L}$:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}.$$

Substituting the three local derivatives

$$\frac{\partial \mathcal{L}}{\partial a} = a - y, \quad \frac{\partial a}{\partial z} = f'(z), \quad \frac{\partial z}{\partial w} = x, \quad \frac{\partial z}{\partial b} = 1,$$

gives the exact parameter derivatives:

Single-neuron gradients:

$$\frac{\partial \mathcal{L}}{\partial w} = (a - y)f'(z)x, \quad \frac{\partial \mathcal{L}}{\partial b} = (a - y)f'(z).$$

9 Worked Single-Neuron Example

Suppose $x = 2$, $w = 0.5$, $b = 0.2$, target $y = 1$, and $f(z) = \sigma(z)$.

1. Forward pass:

$$\begin{aligned} z &= wx + b = (0.5)(2) + 0.2 = 1.2, \\ a &= \sigma(1.2) = \frac{1}{1 + e^{-1.2}} \approx 0.7685, \\ \mathcal{L} &= \frac{1}{2}(a - y)^2 = \frac{1}{2}(0.7685 - 1)^2 \approx 0.0267. \end{aligned}$$

2. Backward pass:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial a} &= a - y = 0.7685 - 1 = -0.2315, \\ \frac{\partial a}{\partial z} &= a(1 - a) \approx (0.7685)(0.2315) \approx 0.1779, \\ \frac{\partial \mathcal{L}}{\partial w} &= (-0.2315)(0.1779)(2) \approx -0.0824, \\ \frac{\partial \mathcal{L}}{\partial b} &= (-0.2315)(0.1779)(1) \approx -0.0412. \end{aligned}$$

Collecting the partial derivatives into a gradient vector:

$$\nabla_{(w,b)} \mathcal{L} = \begin{bmatrix} -0.0824 \\ -0.0412 \end{bmatrix}.$$

Because both gradient components are negative, increasing w and b slightly will decrease the loss.

10 Backpropagation versus Gradient Descent

Backpropagation is reverse-mode automatic differentiation: it computes $\nabla_{\theta} \mathcal{L}$ for all parameters in a single backward pass.

Gradient descent is an optimization step that updates parameters using the computed gradient:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L},$$

where $\eta > 0$ is the learning rate.

11 First-Order Taylor Justification

For a differentiable loss $\mathcal{L}(\theta)$ and a small parameter step $\Delta\theta$, the first-order Taylor expansion is

$$\mathcal{L}(\theta + \Delta\theta) \approx \mathcal{L}(\theta) + \nabla_{\theta} \mathcal{L}(\theta)^T \Delta\theta.$$

Choosing the negative gradient direction $\Delta\theta = -\eta \nabla_{\theta} \mathcal{L}(\theta)$ gives

$$\mathcal{L}(\theta - \eta \nabla_{\theta} \mathcal{L}(\theta)) \approx \mathcal{L}(\theta) - \eta \|\nabla_{\theta} \mathcal{L}(\theta)\|_2^2.$$

Since $\|\nabla_{\theta} \mathcal{L}(\theta)\|_2^2 \geq 0$, moving in the direction $-\nabla_{\theta} \mathcal{L}$ guarantees a local decrease in loss for sufficiently small $\eta > 0$ whenever $\nabla_{\theta} \mathcal{L}(\theta) \neq \mathbf{0}$.

12 Vectorized Backpropagation Equations

Define the error signal (sensitivity vector) at layer l as the derivative of the loss with respect to the pre-activations $\mathbf{z}^{(l)}$:

$$\boldsymbol{\delta}^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{n_l}.$$

Core Backpropagation Equations:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot f'^{(L)}(\mathbf{z}^{(L)}) \quad (\text{Output error}) \quad (1)$$

$$\boldsymbol{\delta}^{(l)} = \left(\left(W^{(l+1)} \right)^T \boldsymbol{\delta}^{(l+1)} \right) \odot f'^{(l)}(\mathbf{z}^{(l)}) \quad (\text{Hidden error recurrence}) \quad (2)$$

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T \quad (\text{Weight gradient}) \quad (3)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \quad (\text{Bias gradient}) \quad (4)$$

13 Deriving the Weight Matrix Outer Product

At layer l , the j -th pre-activation is

$$z_j = \sum_{i=1}^{n_{l-1}} W_{ji} a_i + b_j.$$

Let $\delta_j = \frac{\partial \mathcal{L}}{\partial z_j}$. Applying the chain rule to a single weight entry W_{ji} :

$$\frac{\partial \mathcal{L}}{\partial W_{ji}} = \frac{\partial \mathcal{L}}{\partial z_j} \frac{\partial z_j}{\partial W_{ji}} = \delta_j a_i.$$

Recognizing that $\delta_j a_i$ is the (j, i) entry of the outer product $\delta \mathbf{a}^T$:

$$\left(\delta \mathbf{a}^T \right)_{ji} = \delta_j a_i \implies \nabla_W \mathcal{L} = \delta \mathbf{a}^T \in \mathbb{R}^{n_l \times n_{l-1}}.$$

14 Deriving the Hidden-Layer Recurrence

Consider how $\mathbf{z}^{(l)}$ influences $\mathbf{z}^{(l+1)}$:

$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}), \quad z_k^{(l+1)} = \sum_{j=1}^{n_l} W_{kj}^{(l+1)} a_j^{(l)} + b_k^{(l+1)}.$$

By the multivariable chain rule, each activation $a_j^{(l)}$ influences every unit k in layer $l+1$:

$$\frac{\partial \mathcal{L}}{\partial a_j^{(l)}} = \sum_{k=1}^{n_{l+1}} \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} = \sum_{k=1}^{n_{l+1}} \delta_k^{(l+1)} W_{kj}^{(l+1)}.$$

In matrix notation, summing over the first index k of $W_{kj}^{(l+1)}$ corresponds to multiplying by the transpose $\left(W^{(l+1)}\right)^T$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} = \left(W^{(l+1)}\right)^T \delta^{(l+1)}.$$

Finally, multiplying element-wise by the local activation derivative $\frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} = f'^{(l)}(z_j^{(l)})$ yields

$$\delta^{(l)} = \left(\left(W^{(l+1)}\right)^T \delta^{(l+1)} \right) \odot f'^{(l)}(\mathbf{z}^{(l)}).$$

15 Worked Two-Layer Network by Hand

Consider a $2 \rightarrow 2 \rightarrow 1$ network with ReLU hidden activation, linear output, and squared error loss:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.8 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 0.4 & -0.6 \end{bmatrix}, \quad b^{(2)} = 0.2, \quad y = 0.5.$$

15.1 Forward Pass

Hidden layer pre-activation and ReLU activation:

$$\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 0.5(1) - 0.2(2) + 0.1 \\ 0.3(1) + 0.8(2) - 0.1 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix}, \quad \mathbf{a}^{(1)} = \max(\mathbf{0}, \mathbf{z}^{(1)}) = \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix}.$$

Output layer and scalar loss:

$$\hat{y} = z^{(2)} = W^{(2)}\mathbf{a}^{(1)} + b^{(2)} = (0.4)(0.2) + (-0.6)(1.8) + 0.2 = 0.08 - 1.08 + 0.2 = -0.8,$$

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(-0.8 - 0.5)^2 = \frac{1}{2}(-1.3)^2 = 0.845.$$

15.2 Backward Pass

Output error signal and Layer 2 parameter gradients:

$$\delta^{(2)} = \hat{y} - y = -0.8 - 0.5 = -1.3,$$

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = \delta^{(2)} (\mathbf{a}^{(1)})^T = -1.3 \begin{bmatrix} 0.2 & 1.8 \end{bmatrix} = \begin{bmatrix} -0.26 & -2.34 \end{bmatrix},$$

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^{(2)} = -1.3.$$

Propagate error signal to Hidden Layer 1 (since both components of $\mathbf{z}^{(1)} > 0$, $f'(\mathbf{z}^{(1)}) = [1, 1]^T$):

$$\boldsymbol{\delta}^{(1)} = (W^{(2)})^T \delta^{(2)} \odot f'(\mathbf{z}^{(1)}) = \begin{bmatrix} 0.4 \\ -0.6 \end{bmatrix} (-1.3) \odot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix}.$$

Layer 1 parameter gradients:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^T = \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} -0.52 & -1.04 \\ 0.78 & 1.56 \end{bmatrix}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \boldsymbol{\delta}^{(1)} = \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix}.$$

16 Jacobian and Vector-Jacobian Products

For a vector mapping $\mathbf{y} = f(\mathbf{x})$, the Jacobian matrix $J_f = \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ has entries $(J_f)_{ij} = \frac{\partial y_i}{\partial x_j}$. For a composition $\mathbf{y} = (f \circ g)(\mathbf{x}) = f(g(\mathbf{x}))$, the multivariable chain rule states

$$J_{f \circ g} = J_f J_g.$$

Rather than materializing huge intermediate Jacobian matrices explicitly, backpropagation propagates a row/column sensitivity vector $\mathbf{v} = \nabla_{\mathbf{y}} \mathcal{L}$ backward via **vector-Jacobian products (VJPs)**:

$$\nabla_{\mathbf{x}} \mathcal{L} = J_g^T J_f^T \mathbf{v}.$$

17 Softmax and Cross-Entropy Derivative

For K -class classification with logits $\mathbf{z} \in \mathbb{R}^K$, the softmax function produces a probability vector $\mathbf{p} \in \mathbb{R}^K$:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} = \frac{e^{z_i - m}}{\sum_{j=1}^K e^{z_j - m}}, \quad m = \max_j z_j.$$

Paired with categorical cross-entropy for a one-hot target $\mathbf{y}$ ($\sum_i y_i = 1$):

$$\mathcal{L} = -\sum_{i=1}^K y_i \log p_i = -\sum_{i=1}^K y_i z_i + \log \left(\sum_{j=1}^K e^{z_j} \right).$$

Differentiating with respect to logit z_k yields a clean cancellation:

$$\frac{\partial \mathcal{L}}{\partial z_k} = -y_k + \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} = p_k - y_k.$$

Canonical Softmax + Cross-Entropy Gradient:

$$\nabla_z \mathcal{L} = \mathbf{p} - \mathbf{y}.$$

18 Mini-Batch Gradients and Optimizers

For a mini-batch of m examples, the batch loss and its gradient are the sample averages:

$$\mathcal{L}_{\text{batch}} = \frac{1}{m} \sum_{i=1}^m \mathcal{L}_i, \quad \nabla_{\theta} \mathcal{L}_{\text{batch}} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \mathcal{L}_i.$$

18.1 Standard Parameter Update Rules

- **SGD:** $\theta_{t+1} = \theta_t - \eta \mathbf{g}_t$
- **Momentum:** $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t$, $\theta_{t+1} = \theta_t - \eta \mathbf{v}_t$
- **RMSProp:** $\mathbf{s}_t = \beta \mathbf{s}_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t$, $\theta_{t+1} = \theta_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \epsilon}$
- **Adam:**

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, & \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t, \\ \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1 - \beta_1^t}, & \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1 - \beta_2^t}, & \theta_{t+1} &= \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \end{aligned}$$

19 Numerical Gradient Checking and Pathologies

19.1 Centered Finite Differences

Analytical backpropagation implementations can be verified numerically using centered differences:

$$g_{\text{numeric}} = \frac{\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)}{2\epsilon} + O(\epsilon^2), \quad \text{error} = \frac{|g_{\text{analytic}} - g_{\text{numeric}}|}{\max(1, |g_{\text{analytic}}|, |g_{\text{numeric}}|)}.$$

19.2 Vanishing and Exploding Gradients

Across L layers, the gradient at early layers involves a product of layer Jacobians:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} = J_2^T J_3^T \cdots J_L^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}}.$$

Because $\sigma'(z) \in (0, 1/4]$, deep sigmoid networks shrink gradients exponentially toward zero ($\|\nabla \mathcal{L}\| \rightarrow 0$). ReLU has derivative 1 for all $z > 0$, preserving gradient flow on active paths.

20 Procedure and Practice Exercise

20.1 7-Step Hand-Solving Sequence

1. **Record dimensions:** Write $n_0, n_1, \dots, n_L$ and verify $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$.
2. **Forward pass:** Compute $\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ and $\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)})$ for $l = 1, \dots, L$.
3. **Evaluate loss:** Compute scalar $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$.
4. **Output sensitivity:** Compute $\boldsymbol{\delta}^{(L)} = \partial \mathcal{L} / \partial \mathbf{z}^{(L)}$.
5. **Output layer gradients:** Compute $\nabla_{W^{(L)}} \mathcal{L} = \boldsymbol{\delta}^{(L)}(\mathbf{a}^{(L-1)})^T$ and $\nabla_{\mathbf{b}^{(L)}} \mathcal{L} = \boldsymbol{\delta}^{(L)}$.
6. **Backward recurrence:** Compute $\boldsymbol{\delta}^{(l)} = ((W^{(l+1)})^T \boldsymbol{\delta}^{(l+1)}) \odot f'^{(l)}(\mathbf{z}^{(l)})$ and layer l parameter gradients.
7. **Parameter update:** Apply $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$.

20.2 Practice Problem

For a $2 \rightarrow 2 \rightarrow 1$ network with ReLU hidden layer, linear output, squared error loss, learning rate $\eta = 0.1$, and target $y = 0.8$:

$$\mathbf{x} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 0.4 & -0.7 \\ 0.2 & 0.5 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 0.6 & -0.3 \end{bmatrix}, \quad b^{(2)} = 0.1.$$

Compute by hand: $\mathbf{z}^{(1)}, \mathbf{a}^{(1)}, z^{(2)}, \hat{y}, \mathcal{L}, \boldsymbol{\delta}^{(2)}, \nabla_{W^{(2)}} \mathcal{L}, \nabla_{b^{(2)}} \mathcal{L}, \boldsymbol{\delta}^{(1)}, \nabla_{W^{(1)}} \mathcal{L}, \nabla_{\mathbf{b}^{(1)}} \mathcal{L}$, and the updated parameters after one step.

21 Core Equation Reference

Concept	Equation
Forward Pre-Activation	$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$
Forward Activation	$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)})$
Squared Error Loss	$\mathcal{L} = \frac{1}{2} \ \hat{\mathbf{y}} - \mathbf{y}\ _2^2$
Softmax + Cross-Entropy Gradient	$\nabla_{\mathbf{z}} \mathcal{L} = \mathbf{p} - \mathbf{y}$
Layer Error Signal	$\boldsymbol{\delta}^{(l)} = \partial \mathcal{L} / \partial \mathbf{z}^{(l)}$
Hidden Error Recurrence	$\boldsymbol{\delta}^{(l)} = \left((W^{(l+1)})^T \boldsymbol{\delta}^{(l+1)} \right) \odot f'^{(l)}(\mathbf{z}^{(l)})$
Weight Matrix Gradient	$\nabla_{W^{(l)}} \mathcal{L} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^T$
Bias Vector Gradient	$\nabla_{\mathbf{b}^{(l)}} \mathcal{L} = \boldsymbol{\delta}^{(l)}$
Gradient Descent Update	$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$